Condensation

Marketplace revente de clé
des jeux Steam

Nhat Linh NGUYEN
Corentin DUPAIGNE
Adrien DEU
Jules LADAIGUE

Démonstration

Responsibilities

Corentin Dupaigne
Infrastructure (/infra) & Authentication service (/authentication)

Jules Ladaigue
Backend (/backend)

Nhat Linh Nguyen
UI/UX - Frontend (/frontend)

Adrien Deu
Tests (Unit & e2e tests)

Tech stack

FRONTEND

- Next.js
- TypeScript
- Tailwind CSS

BACKEND

- Spring Boot
- Kafka
- Laravel

DATABASE & CACHE

- PostgreSQL
- PgAdmin

INFRA & OPS

- Docker
- Kubernetes
- Github
- GitHub Actions
- OVH

Git & GitHub Workflow

Branching Strategy

One branch per feature or fix. Prefix naming convention: fix, feat, refactor, infra.

Commit Standards

Maintain atomic commits following the Conventional Commit format (feat, fix, refactor, chore).

Branch Protection

Direct pushes to master are forbidden. Merge to master is only allowed if CI passes.

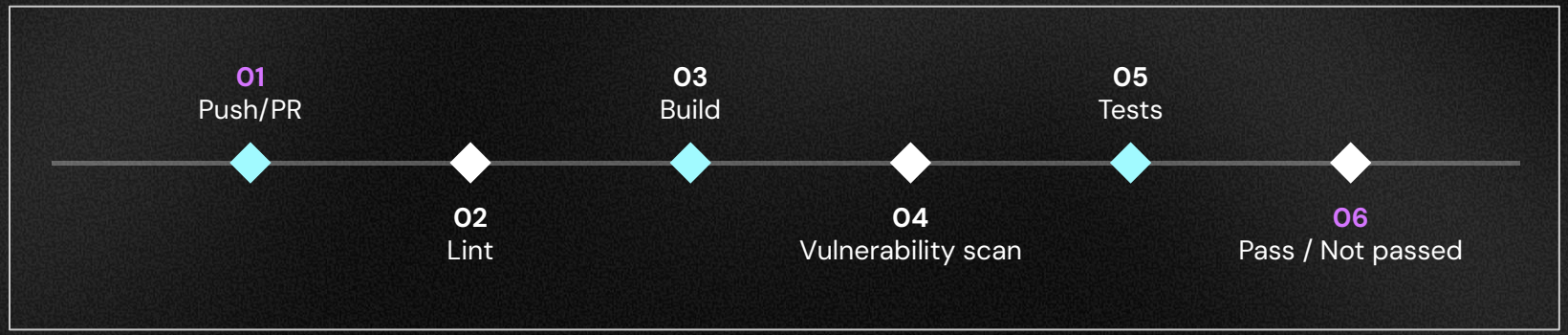
Cleanup

Branches are automatically deleted immediately after they are successfully merged.

1

Devops

Continuous Integration (CI) Pipeline



Continuous Delivery (CD) Pipelines

DEV

- **01** Push to master
- **02** Path filter
- **03** Build:dev
- **04** Push GHCR
- **05** Rollout restart

PROD

- **01** Git tag v*.*.*
- **02** Semver check
- **03** Retag: dev -> v1.2.3
- **05** Set image

2

INFRA & CLUSTER

Infra

Cluster & Hosting

K3s single node, hosted on an OVH vps

Hosts both Dev and Prod environments in isolated namespaces

Networking

Gateway API + Envoy Gateway + MetalLB

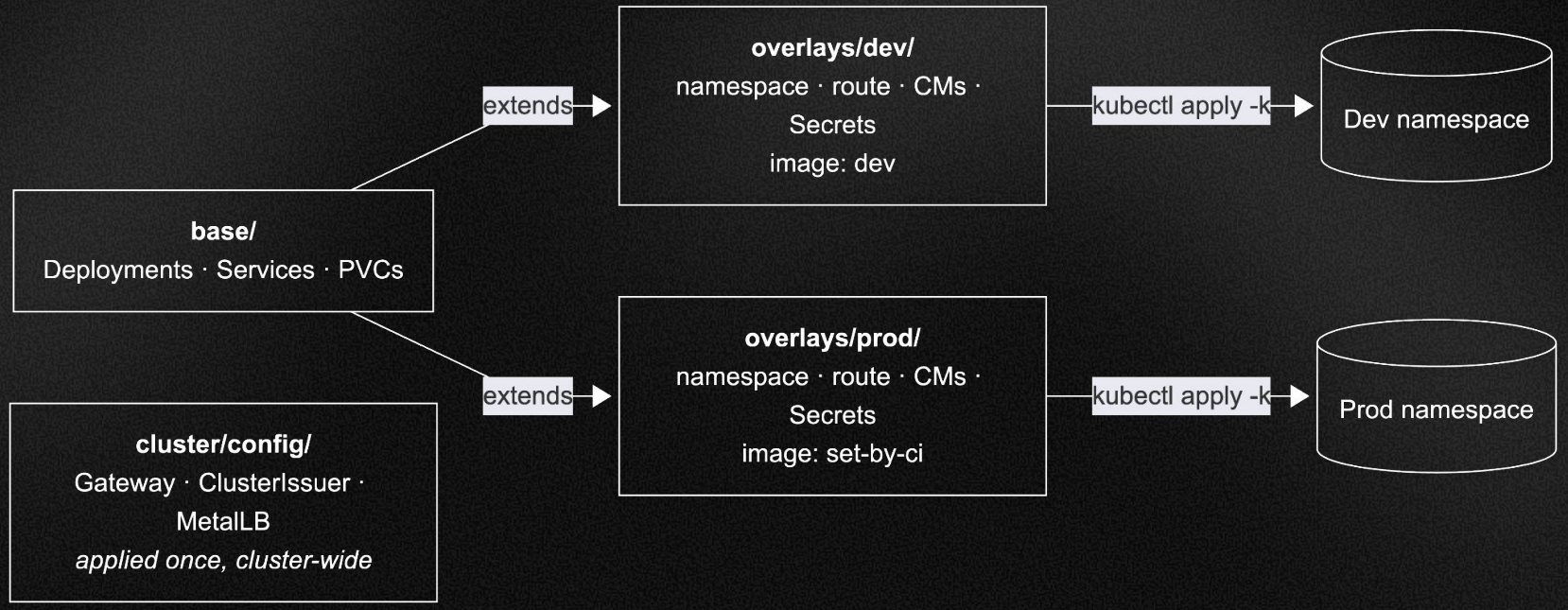
Path-based routing into the cluster, MetalLB exposes a public LB IP

TLS & Certificates

cert-manager + Let's Encrypt

Fully automated issuance and renewal of SSL/TLS certificates for all development and production hostnames.

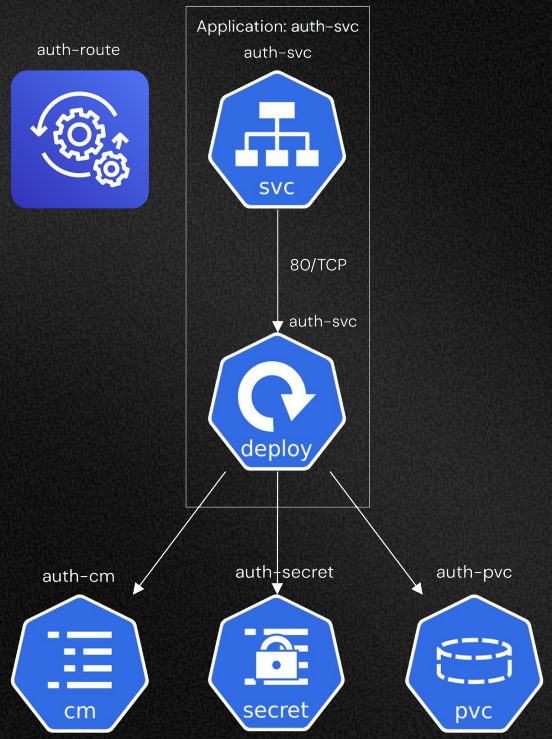
Manifests & Environments



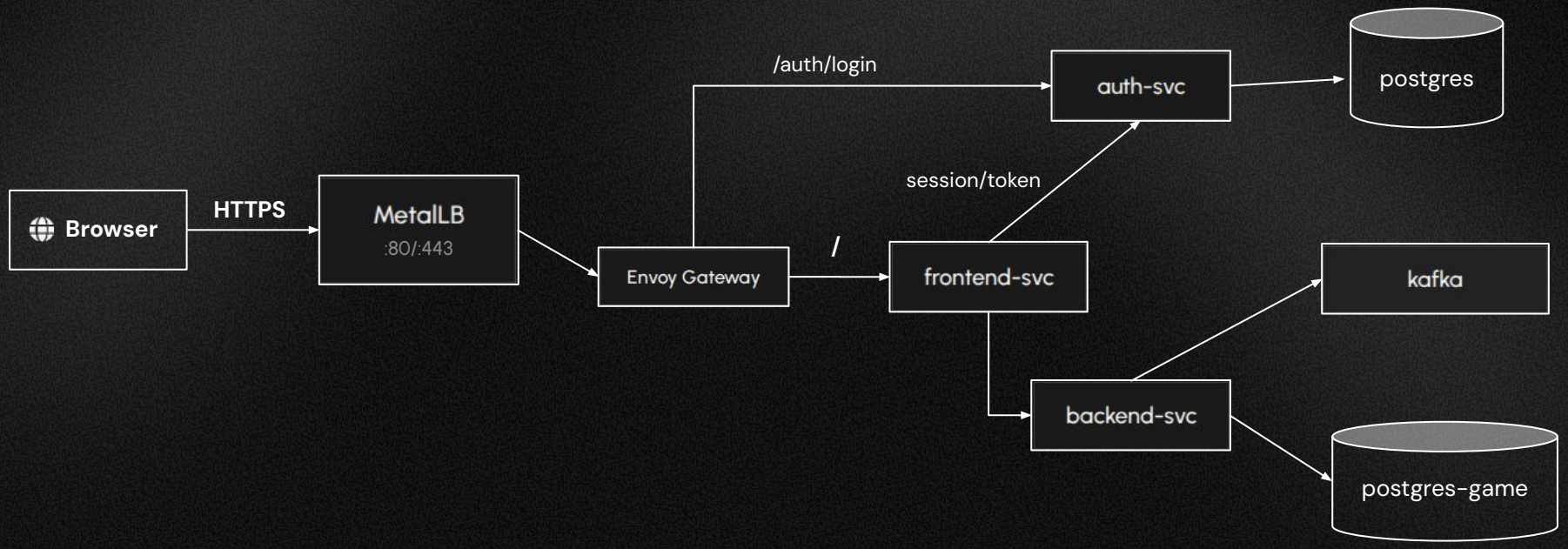
Anatomy of a microservice

Each microservice contains:

- Deployment of n replicas
- Service (expose pod across the cluster)
- Configmaps & Secrets (envs)
- Route (redirect endpoint towards a service)
- PVC? (if persistent volume is necessary)



Path of a request

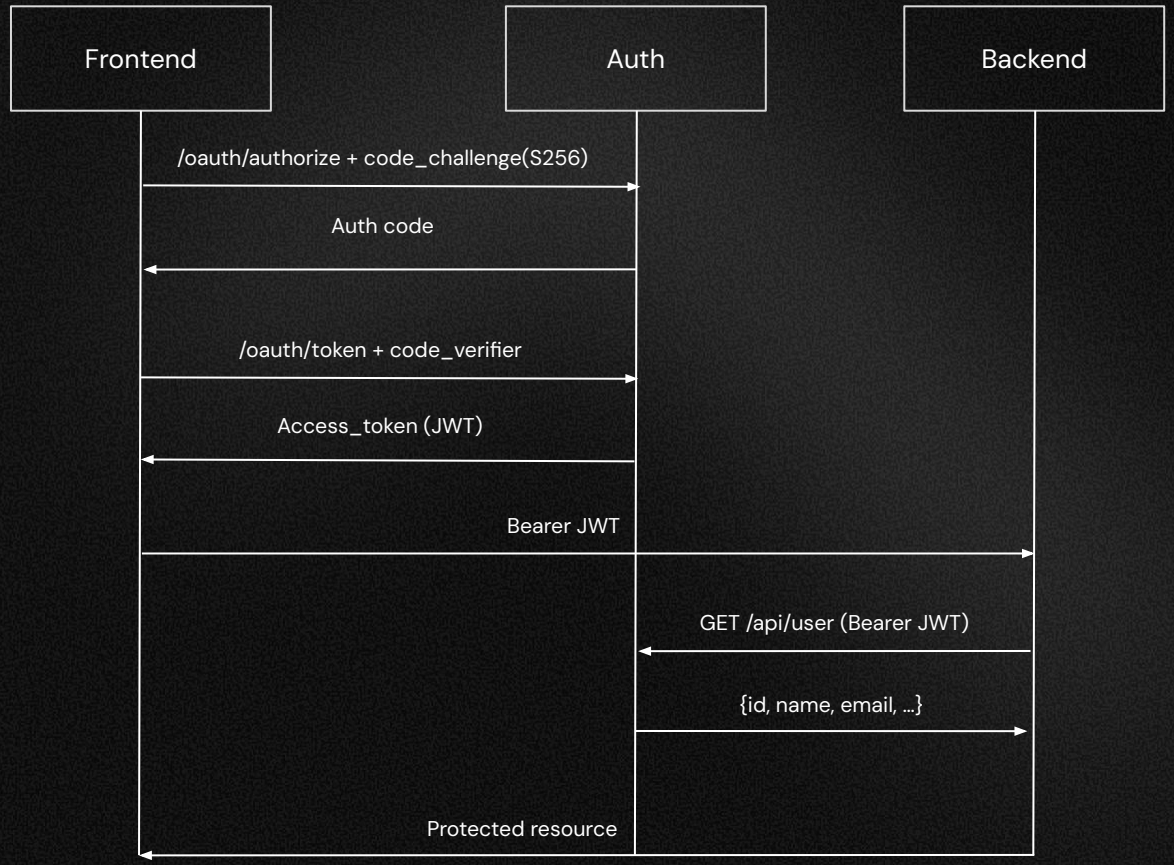


3

AUTHENTICATION

PKCE FLOW

- No client secret required
- RFC 7636 (Oauth 2.1 extension)
- Prevents auth code interception
- Supports unlimited clients
- Enforced by major providers



4

FRONTEND

UI/UX - Design system

COLOR PALETTE

Cyan	Purple	Lime	Surface	Elevated
Primary	Secondary	Tertiary	Component	Card

TYPOGRAPHY

Display LG — 56px	Hero headline
Headline — 24px	Section title
Body — 16px	Default copy
Label MD — 12px	Tags · Sub-header

FONTS

LOREM IPSUM - Space Grotesk

LOREM IPSUM – DM Sans

Next JS - Javascript Framework

Next JS backend proxies

User

GET /games

Frontend Client browser

GET /api/games

Next JS Proxy Server side

GET /games

Backend Spring

File based routing

```
cart
└── page.tsx          /cart

games
└── [id]
    └── page.tsx      /games/ids
    └── page.tsx      /games

settings
└── page.tsx          /settings
```

Tailwind CSS + Bun

Tailwind CSS

Frontend styling system

One tailwind.config for all design tokens

Bun

TS Package manager

bun install vs npm: seconds not minutes

Built-in bundler

Runs .ts natively — no ts-node needed

Stripe

- **Role**
  Handles real-money transactions only — no direct DB writes from Stripe
- **Webhook bridge**
  Stripe calls /api/stripe/webhook → Spring Boot /api/internal/balance secured with INTERNAL_SECRET
- **No card data stored**
  Stripe Checkout handles PCI compliance
- **Balance polling**
  After top-up, frontend polls /api/balance for 15s to confirm credit

User - Wallet top up

Stripe checkout -> Webhook -> Balance funded

User - Direct game purchase (Card)

Stripe checkout -> Webhook -> Balance funded + Order created

User - Pay with balance

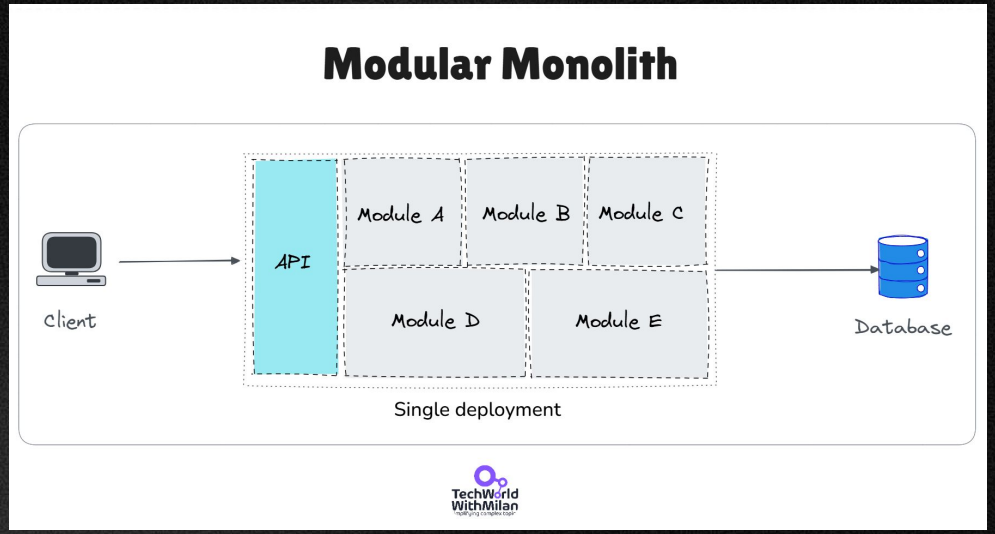
No Stripe but Spring Boot deduce balance and create order directly

5

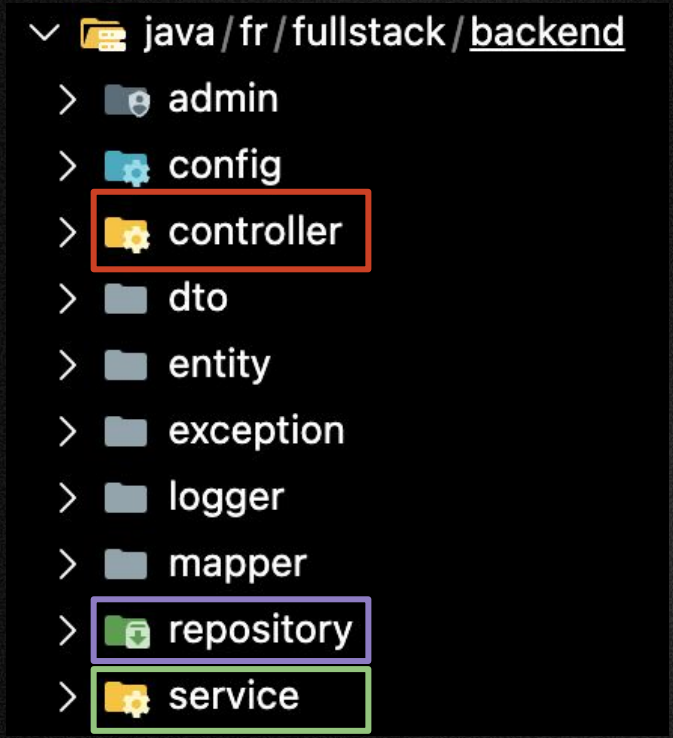
BACKEND

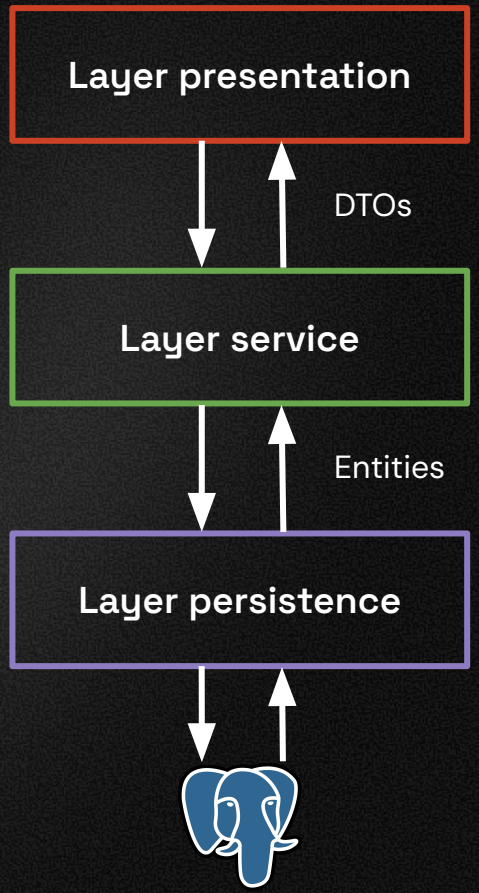
Backend implementation

- **Spring Boot**
  - Standard industriel
  - Robustesse de l'application
  - Écosystème mature
- **Approche Monolith modulaire**
  - Réduction de la complexité
  - Séparation claire des domaines
- **Persistence**
  - JPA & Hibernates
- **Swagger complet**
  - Contrat de l'interface
  - 26 endpoints



Architecture

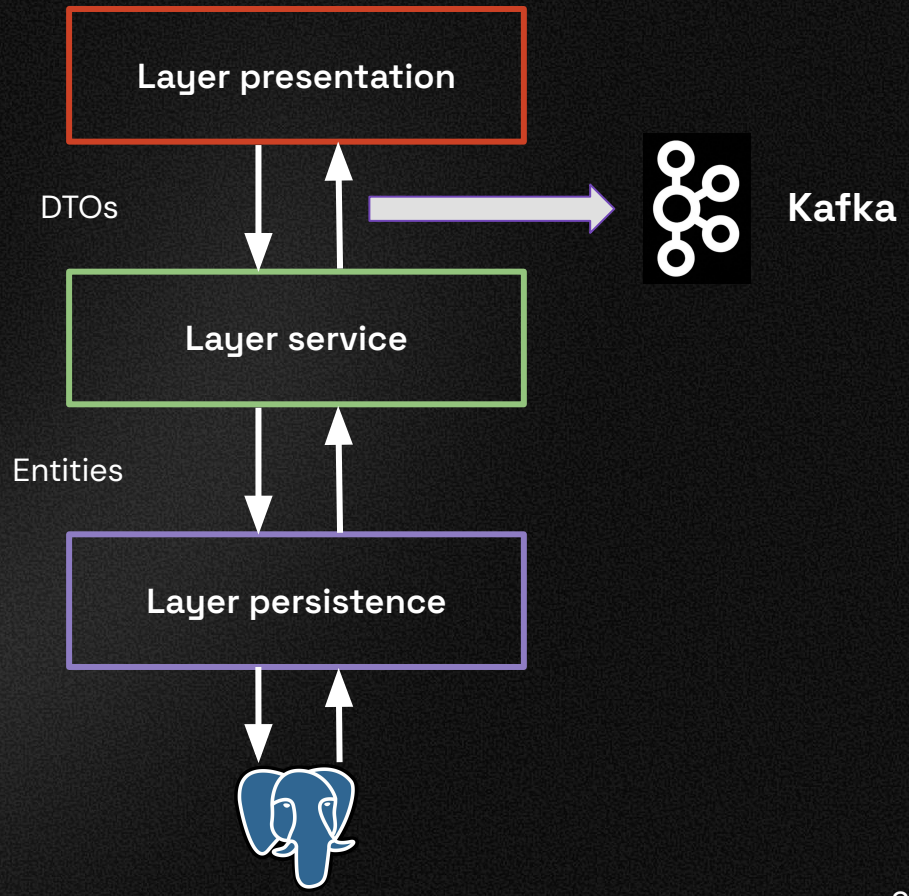




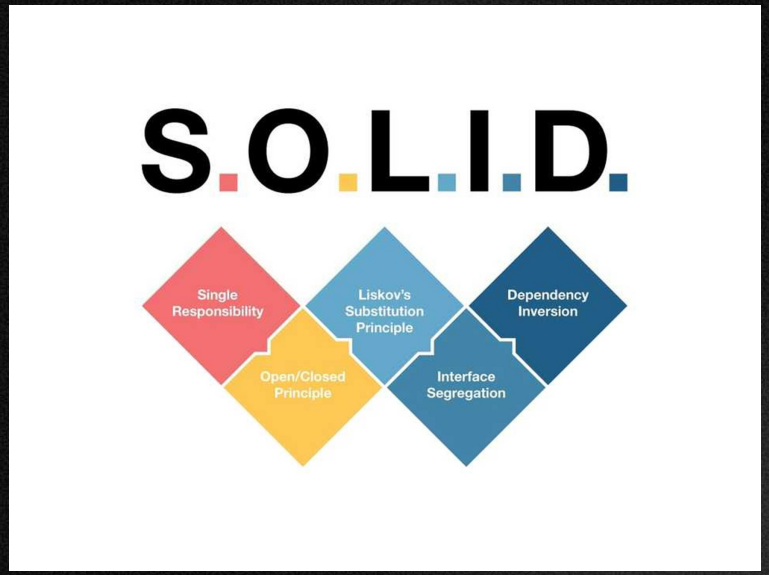
Kafka

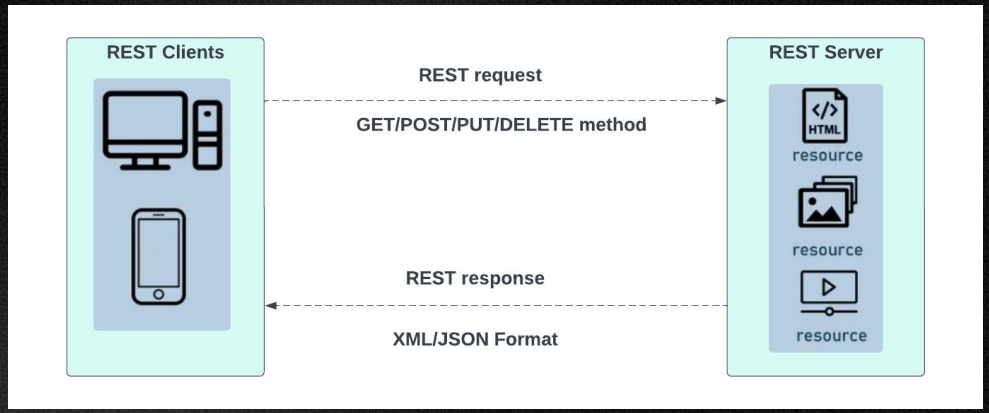
Event driven development

- Stock toutes les sorties services
- Découplage entre les logs et l'application
- Audit immuable
- Restauration de l'application à un état antérieur
- Massivement scalable



Software craftsmanship





6

Tests

Tests — Vue d'ensemble

92	< 5 s	169	~42 min
Unitaires	Durée unit	E2E	Durée e2e

Unit — Framework	JUnit Jupiter 5 · Mockito · AssertJ · MapStruct 1.6.3
Unit — Runtime	Java 17 · Spring Boot 4.0.4 · H2 in-memory (dialecte PostgreSQL)
Unit — Couches	Controllers · Services · DTOs · Entités JPA · Mapper
E2E — Framework	Microsoft.Playwright 1.58 · NUnit 4.5.1 + Playwright.NUnit
E2E — Runtime	C# / .NET 10 · Chromium headless 1920×1080
E2E — Architecture	Page Object Model · 15 fixtures · stack dockerisée complète

Génération du rapport & fast-fail

8 s	8 s	~5,3 s	24 s
FF_TIMEOUT (base)	Navigation Playwright	Expect() polling	Hard cap NUnit

Orchestrateur	run-e2e-with-report.sh — docker compose up · wait_http · seed auth · dotnet test
Pipeline	TRX → trx-to-html.py → report-<TS>.html · cp report-latest.html
Rapport HTML	Monofichier · CSS inline · groupé par fixture · failures dépliables
Dérivation	ExpectTimeout = FF_TIMEOUT × 2/3 · DefaultTimeout = FF_TIMEOUT × 3
Cold start	--no-fast-fail → timeouts larges 30 / 5 / 20 s (build .next vide)

7

Reflections

What could have been better?

- Backend fetches auth public key
- Microservices architecture + communication between services with kafka
- ArgoCD
- Cloud Provider (aws, gcp, ...)
- Contact page implementation
- Loading time optimization

Merci de votre attention